The Transported Probability Density Function (PDF) Method and its Relation to the Black-Scholes-Merton Equation

Christian DiPietrantonio

ME 5311: Computational Methods to Viscous Flows
University of Connecticut
April 29, 2026

Contents

1 Introduction **1**

2 Financial Derivatives and the Black-Scholes-Merton Equation **1**
2.1 Financial Derivatives and Options 1
2.2 The Black-Scholes-Merton Equation 2

3 Derivation of the Black-Scholes-Merton Equation **3**
3.1 Stochastic Calculus and Itô's Lemma 3
3.2 The Stock Price Model: Geometric Brownian Motion 3
3.3 Applying Itô's Lemma to the Option Value 3
3.4 Constructing the Hedged Portfolio 3
3.5 No Arbitrage Condition 3

4 The Transported PDF Method **3**
4.1 Statistical Description of Turbulent Flows 3
4.2 Eulerian PDF Transport Equation 3
4.3 The Generalized Langevin Model 4
4.4 Derivation of the PDF Transport Equation 4

5 The Connection Between the BSM Equation and the Transported PDF Method **4**
5.1 Structural Equivalence of the SDEs 4
5.2 Variable Mapping 4

6 Solution Methods **4**
6.1 Eulerian Methods 4
6.2 Lagrangian Methods 4
6.3 Constant vs. Variable Coefficients 4

7 Computational Experiments and Results **4**
7.1 Problem Description 4
7.2 Analytical Solution 5
7.3 Finite Difference Solver (Eulerian) 5
7.4 Monte Carlo SOlver (Lagrangian) 5
7.5 Results: Option Pricing 5
7.6 Results: Monte Carlo Visualization 5
7.7 Results: PDF Transport 5
7.8 Results: Monte Carlo Convergence 5

8 Conclusions **5**

A Copy of Program Listing **7**

1 Introduction

This paper explores the mathematical connection between the Black-Scholes-Merton (BSM) equation for pricing financial derivatives and the transported probability density function (PDF) method employed in computational fluid dynamics (CFD). The central argument is that these two equations, from completely different fields, are derived from stochastic differential equations (SDEs) of the same dift plus diffusion form, and can be solved using the same classes of numerical methods. The BSM equation is derived from the geometric brownian motion SDE, while the transported PDF equation is derived from the generalized Langevin SDE for the velocity of a fluid particle in turbulence. Even though the two equations were developed in separate disciplines, these two equations share a mathematical connection through stochastic calculus.

The paper begins by establishing the financial definitions and background necessary to the understanding of the further sections. Following this, the BSM equatin is derived following the approach outlined by Hull [1] and Merton [2]. The transporded PDF method is then introduced following the work of Pope [3], and the connection between it and the BSM equation is illustrated through comparison of the underlyinbg SDEs and variable mapping. The paper then outlines the solution methods for the BSM and the transported PDF equation, which include grid-based Eulerian and particle-based Lagrangian approaches. To demonstrate these solution methods, a Python program was developed that implements three different solution methods for pricing a European call option. The first solution method is simply the analytical solution to the BSM equation. Following this, the solver implements an implicit finite difference scheme (Eulerian approach), and a Monte Carlo simulation (Lagrangian approach). The results are compared to one another, and also used to visualize the transport of the probability density function of the stock price. The latter of which can be directly related to the transported PDF of a the velocity of a fluid particle as obtained by the transported PDF method. The benefits and pitfalls of the different solution approaches are also explored. Finally, the paper concludes by discussing the real-world applications of this cross-disciplinary connection, and how advancements in either quantitative finance or computational fluid dynamics have potential to be utilized by the other.

To begin the comparison of the equations and methods, a foundational understanding of the Black-Scholes-Merton equation is necessary. [1]

2 Financial Derivatives and the Black-Scholes-Merton Equation

2.1 Financial Derivatives and Options

In order to understand the Black-Scholes-Merton equation, a foundational understanding of financial derivatives, specifically options, is required. According to Hull, a derivative is a "financial instrument whose value depends on (or derives from) the values of other, more basic, underlying variables" [1]. A stock option, for example, is a derivatve whose value is dependant on the price of a stock. However, different derivatives can be dependant on almost any variable, even the weather. For this paper, the primary derivatives of interest are

options, which are securities that give "the right to buy or sell an asset, subject to certain conditions, within a specified period of time" [4].

The two major types of options are call and put options. A call option gives the holder the right, but not the obligation, to buy the underlying asset by a certain data for a certain price. A put option gives the holder the right, but not the obligation, to sell the underlying asset by a certain data for a certain price [1]. Options can also be further classified by when they can be excercised. American options can be excercised at any time up to the expiration date as specified in the contract, while European options can only be excercised on the expiration date itself [4]. The price at which the asset may be bought or sold is refered to as the strike price, K, and the expiration date can an also be refrered to as the maturity date, T.

For the remainder of this paper, the focus will be on European call options. The value of a European call option at expiry can be determined by the following payoff function:

$$V = \max(S_T - K) \tag{1}$$

Where S_T is the stock price at expiry. From this payoff function one can observe that, if the stock price exceeds the strike price at expiry, the holder profits. However, if the stock price at expiry is below the strike price, the option expires worthless, as the holder is not obligated to execute the option. In this latter scenario, the holder only loses what they paid for the option. The Black-Scholes-Merton equation aims to determine a price for the contract, before its value is known at expiry.

2.2 The Black-Scholes-Merton Equation

In deriving their valuation formula, Black and Scholes [4] assume an idealized, frictionless market with continuous trading, no transaction costs, constant interest rates, no dividends, unrestricted short selling, and the ability to borrow or lend at the risk-free rate. The option is also assumed to be a European call option. Furthermore, an arbitrage-free market is also assumed, which means that no trading strategy can generate riskless. More explicitly, an arbitrage opportunity can be formally defined as "a (self-financing) trading strategy" that starts with zero value and terminates with a positive value. Therefore, a market is arbitrage-free if there are no such arbitrage opportunities [5].

The assumptions outlined above lead to the derivation of the following second-order linear parabolic Partial Differential Equation (PDE) for the price $V(S,t)$ of a European call option. The same equation was independently derived by Robert C. Merton [2] through the use of stochastic calculus, and therefore is commonly referred to as the Black-Scholes-Merton PDE:

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0 \tag{2}$$

Where S is the underlying stock price, t is time, r is the risk-free interest rate, and σ is the volatility of the stock, a measure of how much the stock price fluctuates, or in other words a "measure of our uncertainty about the returns provided by the stock" [1]. This equation was obtained by forming a hedged portfolio that eliminates risk and requiring that its returns equal the risk-free rate in an arbitrage-free market.

It should also be noted that the structure of Equation 2 can be recognized as having the same form of the general transport equation given the context of this course. Where $\frac{\partial V}{\partial t}$ is the temporal term, $\frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2}$ is a diffusion term, $rS\frac{\partial V}{\partial S}$ is an advection term, and rV is a source (in this case, decay) term. This course has dedicated ample time to discritizing and computatonally solving equations of this exact form.

3 Derivation of the Black-Scholes-Merton Equation

The use of Stochastic Calculus in the derivation of Equation 2 is crucial to the argument of this paper. Therefore, this section will walk through such derivation as done so by Hull [1] and Merton [2].

3.1 Stochastic Calculus and Itô's Lemma

TBD

3.2 The Stock Price Model: Geometric Brownian Motion

TBD

3.3 Applying Itô's Lemma to the Option Value

TBD

3.4 Constructing the Hedged Portfolio

TBD

3.5 No Arbitrage Condition

TBD

4 The Transported PDF Method

4.1 Statistical Description of Turbulent Flows

TBD

4.2 Eulerian PDF Transport Equation

TBD

4.3 The Generalized Langevin Model

TBD

4.4 Derivation of the PDF Transport Equation

TBD

5 The Connection Between the BSM Equation and the Transported PDF Method

5.1 Structural Equivalence of the SDEs

TBD

5.2 Variable Mapping

TBD

6 Solution Methods

TBD

6.1 Eulerian Methods

TBD

6.2 Lagrangian Methods

TBD

6.3 Constant vs. Variable Coefficients

TBD

7 Computational Experiments and Results

TBD

7.1 Problem Description

TBD

7.2 Analytical Solution

TBD

7.3 Finite Difference Solver (Eulerian)

TBD

7.4 Monte Carlo SOlver (Lagrangian)

TBD

7.5 Results: Option Pricing

TBD

7.6 Results: Monte Carlo Visualization

TBD

7.7 Results: PDF Transport

TBD

7.8 Results: Monte Carlo Convergence

TBD

8 Conclusions

TBD

References

[1] J. C. Hull, *Options, Futures, and Other Derivatives*, 10th ed. Boston, MA: Pearson, 2022.

[2] R. C. Merton, "Theory of rational option pricing," *Bell Journal of Economics and Management Science*, vol. 4, no. 1, pp. 141–183, 1973.

[3] S. B. Pope, *Turbulent Flows*. Cambridge: Cambridge University Press, 2000.

[4] F. Black and M. Scholes, "The pricing of options and corporate liabilities," *Journal of Political Economy*, vol. 81, no. 3, pp. 637–654, 1973.

[5] M. Baxter and A. Rennie, *Financial Calculus: An Introduction to Derivative Pricing*. Cambridge: Cambridge University Press, 1996.

A Copy of Program Listing

```python
"""
Christian DiPietrantonio
ME 5311: Computational Methods to Viscous Flows
Term Paper
04/19/2026
----------------------------------------------------------------------
Black-Scholes-Merton Solver
----------------------------------------------------------------------

This script demonstrates three approaches to pricing a European call option:
    1. Analytical Black-Scholes formula (closed-form solution)
    2. Monte Carlo simulation (Lagrangian / stochastic particle method)
    3. Finite Difference (Eulerian / grid-based PDE solver)

Methods 2 and 3 are the Lagrangian and Eulerian numerical approaches.
"""

import numpy as np
from scipy.stats import norm
from scipy.linalg import solve_banded
import matplotlib.pyplot as plt
from matplotlib import cm
from mpl_toolkits.mplot3d import Axes3D
import os

# ----------------------------------------------------------------------
# Parameters
# ----------------------------------------------------------------------
K         = 100.0 # Strike Price ($)
r         = 0.05 # Risk-free interest rate (5%)
sigma     = 0.2  # Volatility
T         = 1.0  # Time to expiry (1 year)

# Monte Carlo Settings
N_paths   = 200000     # Number of simulated paths
N_steps   = 252        # Time steps (trading days in a year)
N_S_pts   = 80         # Number of starting prices to evaluate


# Stock price range
S_min = 50
S_max = 200

# Output directory
out_dir = "figures"
os.makedirs(out_dir, exist_ok=True)

# ----------------------------------------------------------------------
# Analytical Black-Scholes Formula
# ----------------------------------------------------------------------
def bs_call(S, K, r, sigma, T):
```

```python
    """
    Analytical Black-Scholes price for a European call option.
    Page 644, Black-Scholes
    Page 91, Baxter-Rennie
    Page 336, Hull

    Parameters
    ----------
    S     : current stock price
    K     : strike price
    r     : risk free rate
    sigma : volatility
    T     : time to expiration

    Returns
    -------
    V     : option prices
    """

    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    V = S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)

    return V

# ---------------------------------------------------------------------
# Monte Carlo Solver - Terminal Value Method
# ---------------------------------------------------------------------
def mc_option_price(S0, K, r, sigma, T, N_paths):
    """
    Price a European call via Monte Carlo using the exact GBM solution. Instead of
        stepping through time, we jump diretly to S(T). This works because we only need
        the terminal stock price to compute the payoff. For the PDF evolution, we must use
         the full time-steppind version below.

    Page 217, Hull (Long position European Call option payoff)
    Page 336, Hull (Option expected value discounted at risk-free rate)
    Page 471, Hull (Stock price at expiration time)
    Page 473, Hull (Standard Error of estimate)


    Parameters
    ----------
    S0     : starting stock price
    K      : strike price
    r      : risk free rate
    sigma  : volatility
    T      : time to expiration
    N_paths : number of simulated paths

    Returns
    -------
    V_mean : estimated option price/value
    V_std  : standard error of the estimate
```

```python

    """

    Z  = np.random.standard_normal(N_paths)
    ST = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * np.sqrt(T) * Z)

    # payoff for a European call
    payoffs = np.maximum(ST - K, 0.0)

    # discount back to present value
    V_mean = np.exp(-r * T) * np.mean(payoffs)
    V_std  = np.exp(-r * T) * np.std(payoffs) / np.sqrt(N_paths)

    return V_mean, V_std

# ----------------------------------------------------------------------
# Monte Carlo Solver - Full Path Simulation
# ----------------------------------------------------------------------
def mc_full_paths(S0, r, sigma, T, N_paths, N_steps):
    """
    Simulate full GBM paths.

    Page 470, Hull (Discritizing SDEs)
    Note:
        - For this example we use the more accurate analytical solution for the GBM SDE.
            For the transported PDF method, we would discritize the SDE itself.
    Page 471, Hull (Stock price at time step)

    Parameters
    ----------
    S0     : starting stock price
    r      : risk free rate
    sigma  : volatility
    T      : time to expiration
    N_paths : number of simulated paths
    N_steps : number of time steps

    Returns
    -------
    S      : all price paths
    t      : time array

    """

    dt = T / N_steps
    t = np.linspace(0, T, N_steps + 1)

    S = np.zeros((N_steps + 1, N_paths))
    S[0, :] = S0

    # generate random increments
    Z = np.random.standard_normal((N_steps, N_paths))

    # step through time
```

```python
    for i in range(N_steps):
        S[i+1, :] = S[i, :] * np.exp((r - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * Z[i, :])

    return S, t

# ---------------------------------------------------------------------
# Finite Difference Solver (Eulerian / Grid Based PDE Solver)
# ---------------------------------------------------------------------
def fd_bsm(K, r, sigma, T, S_max=500.0, M=500, N_t=5000):
    """
    Solver the BSM PDE using implicit finite difference method.

    This is the Eulerian approach: discritize stock price into a grid and solve the PDE directly at each point. Important to note that we march backwards in time from t = T (expiry) to t=0 (today).

    Parameters
    ----------
    K      : strike price
    r      : risk free rate
    sigma  : volatility
    T      : time to expiration
    S_max  : upper bound of stock price grid
    M      : number of spacial intervals
    N_t    : number of time steps

    Returns
    -------
    S_grid : stock price at grid points
    V      : option values at t = 0 (today)
    """

    # ----- Grid Setup -----
    dS     = S_max / M        # spatial step size
    dt     = T / N_t          # time step size
    S_grid = np.linspace(0, S_max, M + 1)

    # ----- Terminal Condition -----
    # At expiry, option value = payoff = max(S - K, 0)
    V = np.maximum(S_grid - K, 0.0)

    # ----- Build Tridiagonal Coefficients for Interior Nodes j = 1, ..., M - 1 -----
    j = np.arange(1, M)

    a_j = 0.5 * dt * (r * j - sigma**2 * j**2)
    b_j = 1.0 + dt * (sigma**2 * j**2 + r)
    c_j = -0.5 * dt * (sigma**2 * j**2 + r * j)

    # ----- Format into Banded System -----
    n_interior = M - 1 # number of unknowns
    # Banded matrix (l + u + 1, n) where l = 1 lower diagonal, u = 1 upper diagonal
    # element[row, col] maps to ab[u + row - col, col]
    ab = np.zeros((3, n_interior))
```

```python
    ab[0, 1:] = c_j[:-1]
    ab[1,:]   = b_j
    ab[2, :-1] = a_j[1:]

    # ----- Time Marching -----
    for n in range(N_t):
        rhs = V[1:M].copy()

        time_to_expiry = (n + 1) * dt
        V_top = S_max - K * np.exp(-r * time_to_expiry)

        rhs[-1] -= c_j[-1] * V_top

        # Solve the Trigiagonal System
        V[1:M] = solve_banded((1, 1), ab, rhs)

        # Apply boundary condtions to full solution array
        V[0] = 0.0
        V[M] = V_top

    return S_grid, V

# ----------------------------------------------------------------------
# Figure 1: V(S) Comparison
# ----------------------------------------------------------------------
def plot_option_value_curve():
    """
    Compare Monte Carlo, Finite Difference, and Analytical Option Prices.
    """

    print("Computing V(S) curve ...")
    S_arr = np.linspace(S_min, S_max, N_S_pts)

    #Analytical Solution
    V_analytical = bs_call(S_arr, K, r, sigma, T)

    # Monte Carlo (loop over starting prices)
    V_mc        = np.zeros(N_S_pts)
    V_err       = np.zeros(N_S_pts)

    for i, S0 in enumerate(S_arr):
        V_mc[i], V_err[i] = mc_option_price(S0, K, r, sigma, T, N_paths)

    # Finite Difference
    print("    Running finite difference solver...")
    S_fd, V_fd = fd_bsm(K, r, sigma, T, S_max=500.0, M=500, N_t=5000)

    # Interpolate FD results onto same S_arr for comparison
    V_fd_interp = np.interp(S_arr, S_fd, V_fd)

    # ----- Plot -----
    fig, ax = plt.subplots(figsize=(9, 6))

    # Analytical Solution
```

```python
    ax.plot(S_arr, V_analytical, 'k-', linewidth = 2, label='Analytical (Black-Scholes)')

    # Finite Difference Solution
    ax.plot(S_arr, V_fd_interp, 's', color='tab:red', markersize=4, markerfacecolor='none', markeredgewidth=1.0, label='Finite Difference (Eulerian)', alpha=0.8)

    # Monte Carlo (with error bars)
    ax.errorbar(S_arr, V_mc, yerr=4*V_err, marker='o', color='tab:blue', markersize=4, linewidth =1.0, capsize=2, label=f'Monte Carlo / Lagrangian (N = {N_paths:,})', alpha=0.8)

    # Intrinsic Value for Reference
    intrinsic = np.maximum(S_arr - K, 0)
    ax.plot(S_arr, intrinsic, '--', color='tab:gray', linewidth=1.0, alpha=0.8, label='Intrinsic Value: max(S - K, 0)')

    ax.set_xlabel('Current Stock Price, S ($)', fontsize=12)
    ax.set_ylabel('European Call Option Value, V ($)', fontsize=12)
    ax.set_title('European Call Option Pricing: Comparison of Methods', fontsize=15)
    ax.legend(fontsize=12)
    ax.set_xlim(S_min, S_max)
    ax.set_ylim(bottom=-2)
    ax.grid(True, alpha=0.3)

    param_text = (fr'K = \${K:.0f}, r = {r:.0%}, $\sigma$ = {sigma:.0%}, T = {T:.1f} yr')
    ax.text(0.02, 0.97, param_text, transform=ax.transAxes, fontsize=10, verticalalignment='top', bbox=dict(boxstyle='round', facecolor='wheat', edgecolor='black', pad=0.5, alpha=0.8))

    fig.tight_layout()
    fig.savefig(f'{out_dir}/figure01_option_value_curve.png', dpi=200)
    plt.close(fig)
    print(f'    Saved: {out_dir}/figure01_option_value_curve.png')

    return S_arr, V_analytical, V_mc, V_err, V_fd_interp

# ----------------------------------------------------------------------
# Figure 2: Sample GBM Paths
# ----------------------------------------------------------------------
def plot_sample_paths(S0=100.0, N_show=30, N_sim=100):
    """
    Show a handful of GBM trajectories to illustrate stochastic behavior.

    Page 323, 325, Hull (Expected Return)
    """

    print("Simulating sample paths...")
    S, t = mc_full_paths(S0, r, sigma, T, N_sim, N_steps)

    fig, ax = plt.subplots(figsize=(9, 6))

    # plot subset of paths
    for j in range(N_show):
        ax.plot(t, S[:, j], linewidth=0.5, alpha=0.6)
```

```python

    # overlay the mean of the simulated paths
    S_mean_sim = np.mean(S, axis=1)
    ax.plot(t, S_mean_sim, color='tab:blue', linewidth=2.0, alpha=0.9, label=f'Mean of {N_sim} paths simulated')

    # overlay the expected return
    S_expected = S0 * np.exp(r * t)
    ax.plot(t, S_expected, 'k--', linewidth=2.0, label=f'Expected Stock Price: $S_0 e^{{rt}}$')

    ax.axhline(K, color='tab:red', linestyle=':', linewidth=1.5, label=f'Strike Price K = ${K:.0f}')

    ax.set_xlabel('Time (years)', fontsize=12)
    ax.set_ylabel('Stock Price, S ($)', fontsize=12)
    ax.set_title(fr'Geometric Brownian Motion: {N_show} Sample Paths ($S_0$ = \${S0:.0f})', fontsize=15)
    ax.legend(fontsize=12, loc='upper center', bbox_to_anchor=(0.6, 1.0))
    ax.set_xlim(0, T)
    ax.grid(True, alpha=0.3)

    param_text = (fr'r = {r:.0%}, $\sigma$ = {sigma:.0%}, T = {T:.1f} yr')
    ax.text(0.02, 0.97, param_text, transform=ax.transAxes, fontsize=12, verticalalignment='top', bbox=dict(boxstyle='round', facecolor='wheat', edgecolor='black', pad=0.5, alpha=0.8))

    fig.tight_layout()
    fig.savefig(f'{out_dir}/figure02_GMB_sample_paths.png', dpi=200)
    plt.close(fig)
    print(f'    Saved: {out_dir}/figure02_GMB_sample_paths.png')

    return S, t

# ----------------------------------------------------------------------
# Figure 3: PDF Evolution - 3D Surface
# ----------------------------------------------------------------------

def plot_pdf_evolution(S0=100.0, N_sim=200000):
    """
    Visualize how the probability density of a stock price evolves over time using Monte Carlo particle data.
    Note:
        - The exact GBM sol could be used to construct the PDF using the PDF function for a lognormal distribution. However, here the PDF is constructed using MC data, because no PDF function is avalable for the Langevin SDE.

    Page 322, Hull (Mean and Standard Deviation of ln ST)
    Page 84, Montgomery, Runger, Hubele
    """

    print("Computing PDF evolution surface from Monte Carlo data...")

    t_snapshots = np.array([0.01, 0.025, 0.05, 0.075, 0.1, 0.2, 0.4, 0.6, 0.8, 1.0])
```

```
    S_bins = np.linspace(40, 250, 500)
    S_bin_centers = 0.5 * (S_bins[:-1] + S_bins[1:])

    # Simulate paths
    S_all, t_all = mc_full_paths(S0, r, sigma, T, N_sim, N_steps)

    # Build PDF at each time snapshot
    PDF_mc = np.zeros((len(t_snapshots), len(S_bin_centers)))
    for i, t_val in enumerate(t_snapshots):
        idx = np.argmin(np.abs(t_all - t_val))
        counts, _ = np.histogram(S_all[idx, :], bins=S_bins, density=True)
        PDF_mc[i, :] = counts

    fig = plt.figure(figsize=(9, 6))
    ax = fig.add_subplot(111, projection='3d')
    T_mesh, S_mesh = np.meshgrid(t_snapshots, S_bin_centers, indexing='ij')
    surf = ax.plot_surface(S_mesh, T_mesh, PDF_mc, cmap='viridis', alpha=0.85, rstride=1,
         cstride=3, edgecolor='none')

    ax.set_xlabel('Stock Price, S ($)', fontsize=12, labelpad=10)
    ax.set_ylabel('Time (years)', fontsize=12, labelpad=10)
    ax.set_zlabel('Probability Density', fontsize=12, labelpad=10)
    ax.set_title('PDF Evolution from Monte Carlo Particle Data', fontsize=15)
    ax.view_init(elev=20, azim=-45)
    fig.colorbar(surf, ax=ax, shrink=0.45, aspect=15, label ='Probability Density', pad
        =0.10)

    fig.tight_layout()
    fig.savefig(f'{out_dir}/figure03_3d_pdf_evolution.png', dpi=200)
    plt.close(fig)
    print(f'    Saved: {out_dir}/figure03_3d_pdf_evolution.png')
    return t_snapshots, S_bin_centers, PDF_mc

# ----------------------------------------------------------------------
# Figure 4: PDF Evolution - 2D Slices
# ----------------------------------------------------------------------
def plot_pdf_slices(S0=100.0, N_sim=200000):
    """
    Monte Carlo histograms overlaid with analytical log-normal curves. This deomstrates
        that the Lagrangian statistics converge to the Eulerian PDF.

    Page 322, Hull (Mean and Standard Deviation of ln ST)
    Page 84, Montgomery, Runger, Hubele
    """

    print("Computing PDF time slices...")

    t_slices = [0.02, 0.1, 0.25, 0.5, 1.0]
    S_grid  = np.linspace(40, 250, 500)

    # Simulate paths
    S_all, t_all = mc_full_paths(S0, r, sigma, T, N_sim, N_steps)

    colors = cm.plasma(np.linspace(0.1, 0.9, len(t_slices)))
```

```python

    fig, ax = plt.subplots(figsize=(9, 6))
    pdf_max = 0.0

    for i, t_val in enumerate(t_slices):
        # Monte Carlo Histogram
        idx = np.argmin(np.abs(t_all - t_val))
        ax.hist(S_all[idx, :], bins=120, range=(40, 250), density=True, alpha=0.5, color=colors[i], edgecolor='none')

        # Analytical log-normal PDF
        mu_ln = np.log(S0) + (r - 0.5 * sigma**2) * t_val
        sigma_ln = sigma * np.sqrt(t_val)
        pdf_vals = (1.0 / (S_grid * sigma_ln * np.sqrt(2*np.pi)) * np.exp(-0.5 * ((np.log(S_grid) - mu_ln) / sigma_ln)**2))
        pdf_max = max(pdf_max, np.max(pdf_vals))
        ax.plot(S_grid, pdf_vals, color=colors[i], linewidth=2.0, label=f't = {t_val:.2f} years')

    ax.axvline(S0, color='black', linestyle=':', linewidth=2.0, alpha=0.5, label=fr'$S_0$ = \${S0:.0f}')

    ax.set_xlabel('Stock Price, S ($)', fontsize=12)
    ax.set_ylabel('Probability Density', fontsize=12)
    ax.set_title('PDF Evolution: Monte Carlo Histograms vs. Analytical', fontsize=15)
    ax.legend(fontsize=12, loc='upper right')
    ax.set_xlim(40, 250)
    ax.set_ylim(0, pdf_max * 1.1)
    ax.grid(True, alpha=0.3)

    param_text = (fr'$S_0$ = \${S0:.0f}, r = {r:.0%}, $\sigma$ = {sigma:.0%}, N = {N_sim:,} paths')
    ax.text(0.02, 0.97, param_text, transform=ax.transAxes, fontsize=12, verticalalignment='top', bbox=dict(boxstyle='round', facecolor='wheat', edgecolor='black', pad=0.5, alpha=0.8))

    fig.tight_layout()
    fig.savefig(f'{out_dir}/figure04_2D_pdf_slices.png', dpi=200)
    plt.close(fig)
    print(f'    Saved: {out_dir}/figure04_2D_pdf_slices.png')

# ----------------------------------------------------------------------
# Figure 5: Monte Carlo Convergence
# ----------------------------------------------------------------------
def plot_convergance(S0=100.0):
    """
    Show how the MC estimate converges as more paths are simulated.

    Page 473, Hull (MC Standard Error/Confidence intervals)
    """

    print("Computing convergence study...")
    V_exact    = bs_call(S0, K, r, sigma, T)
    N_range    = np.logspace(2, 6, 30).astype(int)
```

```python
    V_estimates = np.zeros(len(N_range))
    V_errors   = np.zeros(len(N_range))

    np.random.seed(11)
    for i, N in enumerate(N_range):
        V_estimates[i], V_errors[i] = mc_option_price(S0, K, r, sigma, T, N)

    fig, ax = plt.subplots(figsize=(9, 6))
    ax.semilogx(N_range, V_estimates, 'o-', color='tab:blue', markersize=5, linewidth=1.0, label='MC Estimate')
    ax.fill_between(N_range, V_estimates - 2*V_errors, V_estimates + 2*V_errors, alpha=0.2, color='tab:blue', label=r'$\pm 2 \sigma$ (95% CI)')
    ax.axhline(V_exact, color='tab:red', linestyle='--', linewidth=2.0, label=f'Exact (BS) = ${V_exact:.2f}')

    ax.set_xlabel('Number of Monte Carlo Paths, N', fontsize=12)
    ax.set_ylabel('Estimated Option Value, V ($)', fontsize=12)
    ax.set_title(fr'Monte Carlo Convergence ($S_0$ = \${S0:.0f})', fontsize=15)
    ax.legend(fontsize=12)
    ax.grid(True, alpha=0.3)

    fig.tight_layout()
    fig.savefig(f'{out_dir}/figure05_MC_convergence.png', dpi=200)
    plt.close(fig)
    print(f'    Saved: {out_dir}/figure05_MC_convergence.png')

# ----------------------------------------------------------------------
# Figure 6: PDF Heatmap
# ----------------------------------------------------------------------
def plot_pdf_heatmap(S0=100.0):
    """
    2D Heatmap: x = stock price, y = time, color = probability density.

    Page 84, Montgomerey, Runger, Hubele (Mean and Varince in Price Space)
    """

    print("Computing PDF heatmap...")
    t_grid  = np.linspace(0.05, T, 500)
    S_grid  = np.linspace(40, 250, 500)

    PDF = np.zeros((len(t_grid), len(S_grid)))
    for i, t_val in enumerate(t_grid):
        mu_ln     = np.log(S0) + (r - 0.5 * sigma**2) * t_val
        sigma_ln  = sigma * np.sqrt(t_val)
        PDF[i, :]  = (1.0 / (S_grid * sigma_ln * np.sqrt(2*np.pi)) * np.exp(-0.5 * ((np.log(S_grid) - mu_ln) / sigma_ln)**2))

    fig, ax = plt.subplots(figsize=(9, 6))
    S_mesh, t_mesh = np.meshgrid(S_grid, t_grid)
    pcm = ax.pcolormesh(S_mesh, t_mesh, PDF, cmap='inferno', shading='gouraud')

    S_expected = S0 * np.exp(r * t_grid)
    ax.plot(S_expected, t_grid, 'w--', linewidth=2.0, label=r'$E(S_T) = S_0 e^{rt}$')

```

```python
    for sign, lbl in [(1, r'+1$\sigma$'), (-1, r'-1$\sigma$')]:
        mu_ln     = np.log(S0) + (r - 0.5 * sigma**2) * t_grid
        sigma_ln  = sigma * np.sqrt(t_grid)
        S_bound   = np.exp(mu_ln + sign * sigma_ln)
        ax.plot(S_bound, t_grid, 'w:', linewidth=1.0, label=lbl, alpha=0.7)

    ax.set_xlabel('Stock Price, S ($)', fontsize=12)
    ax.set_ylabel('Time (years)', fontsize=12)
    ax.set_title('PDF Evolution Heatmap\n' \
                 fr'($S_0$ = \${S0:.0f}, r = {r:.0%}, $\sigma$ = {sigma:.0%})', fontsize=15)
    ax.legend(fontsize=12, loc='upper right', facecolor='black', edgecolor='white', labelcolor='white')
    fig.colorbar(pcm, ax=ax, label ='Probability Density')
    ax.set_xlim(40, 250)

    fig.tight_layout()
    fig.savefig(f'{out_dir}/figure06_pdf_heatmap.png', dpi=200)
    plt.close(fig)
    print(f'    Saved: {out_dir}/figure06_pdf_heatmap.png')

# ----------------------------------------------------------------------
# Main Driver
# ----------------------------------------------------------------------
def main():
    print("=" * 120)
    print("BSM Solver - Monte Carlo + Finite Difference + Analytical")
    print("=" * 120)
    print(fr'Parameters: K=${K}, r={r:.2%}, $\sigma$={sigma:.2%}, T={T} years')
    print(f'Monte Carlo: {N_paths:,} paths, {N_steps} time steps')
    print("=" * 120)

    np.random.seed(2026)

    S_arr, V_an, V_mc, V_err, F_fd = plot_option_value_curve()
    S_paths, t_arr              = plot_sample_paths()
    t_snap, S_centers, PDF_mc   = plot_pdf_evolution()
    plot_pdf_slices()
    plot_convergance()
    plot_pdf_heatmap()

    print("\n" + "=" * 120)
    print("V(S) at selected stock prices")
    print("=" * 120)
    print(f'{'S ($)':>8} {'Analytical':>12} {'Monte Carlo':>12} {'Finite Difference':>12} {'MC Error':>8}')
    print("-" * 120)

    S_fd_check, V_fd_check = fd_bsm(K, r, sigma, T, S_max=500.0, M=500, N_t=5000)
    check_prices = [60, 80, 100, 120, 140, 160]
    for S0 in check_prices:
        V_an_val         = bs_call(S0, K, r, sigma, T)
        V_mc_val, V_mc_err = mc_option_price(S0, K, r, sigma, T, N_paths)
        V_fd_vall        = np.interp(S0, S_fd_check, V_fd_check)
```

```
        print(f'{S0:>8.0f} {V_an_val:>12.2f} {V_mc_val:>12.2f} {V_fd_vall:>12.2f} {V_mc_err:>12.2f}')

    print('\nAll figures saved to:', out_dir)
    print('Done!')

if __name__ == "__main__":
    main()
```